Understanding MCP

The Model Context Protocol, end to end

MCP Client vs MCP Server - and how data flows

A live example: querying a trading bot on a remote VM

Product Training Material

1. What is MCP?

MCP (Model Context Protocol) is an open standard that lets an AI assistant talk to outside tools and data in one consistent way. Think of it as a universal adapter - like USB-C for AI. Instead of hand-coding a custom integration for every app, every database, and every model, each side speaks the same protocol and they just plug together.

The problem it solves

Without a standard, connecting M AI apps to N data sources means building M x N one-off integrations. MCP turns that into M + N: each AI app ships a client once, each data source ships a server once, and any client can talk to any server.

> **In one sentence**
>
> MCP standardises HOW an AI app asks for tools and data, so the same little 'server' you write can be reused by any MCP-aware app - and your app can use any MCP server without custom glue.

The three roles you'll hear about

- Host: the AI application you interact with (e.g. Claude Code). It hosts the model and one or more clients.
- Client: a connector living inside the host. It opens a connection to ONE server, discovers its tools, and calls them on the model's behalf.
- Server: a small program that exposes tools/data over the protocol. It does the actual work when a tool is called.

2. MCP Client vs MCP Server

This is the distinction to anchor on. The client is the CALLER; the server is the DOER. They have a strict 1-to-1 connection and talk in structured messages (JSON-RPC), never free text.

	MCP CLIENT	MCP SERVER
Lives in	The AI app (the host) - e.g. Claude Code	A standalone program you run - e.g. mcp_trades_server.py
Job	Discover tools, decide when to call them, send the call	Receive the call, do the work, return a result
Knows about	Many possible servers	Only its own tools + data
Our setup	Built into Claude Code on your Mac	The 'trades' server that reads trades.db
Initiates?	Yes - the client always starts the conversation	No - it waits and responds

What a server actually exposes

- Tools: functions the model can call (with typed inputs). This is what we use.
- Resources: read-only data the model can pull in for context (files, records).
- Prompts: reusable prompt templates the server offers. (Not used here.)

Our server exposes three TOOLS:

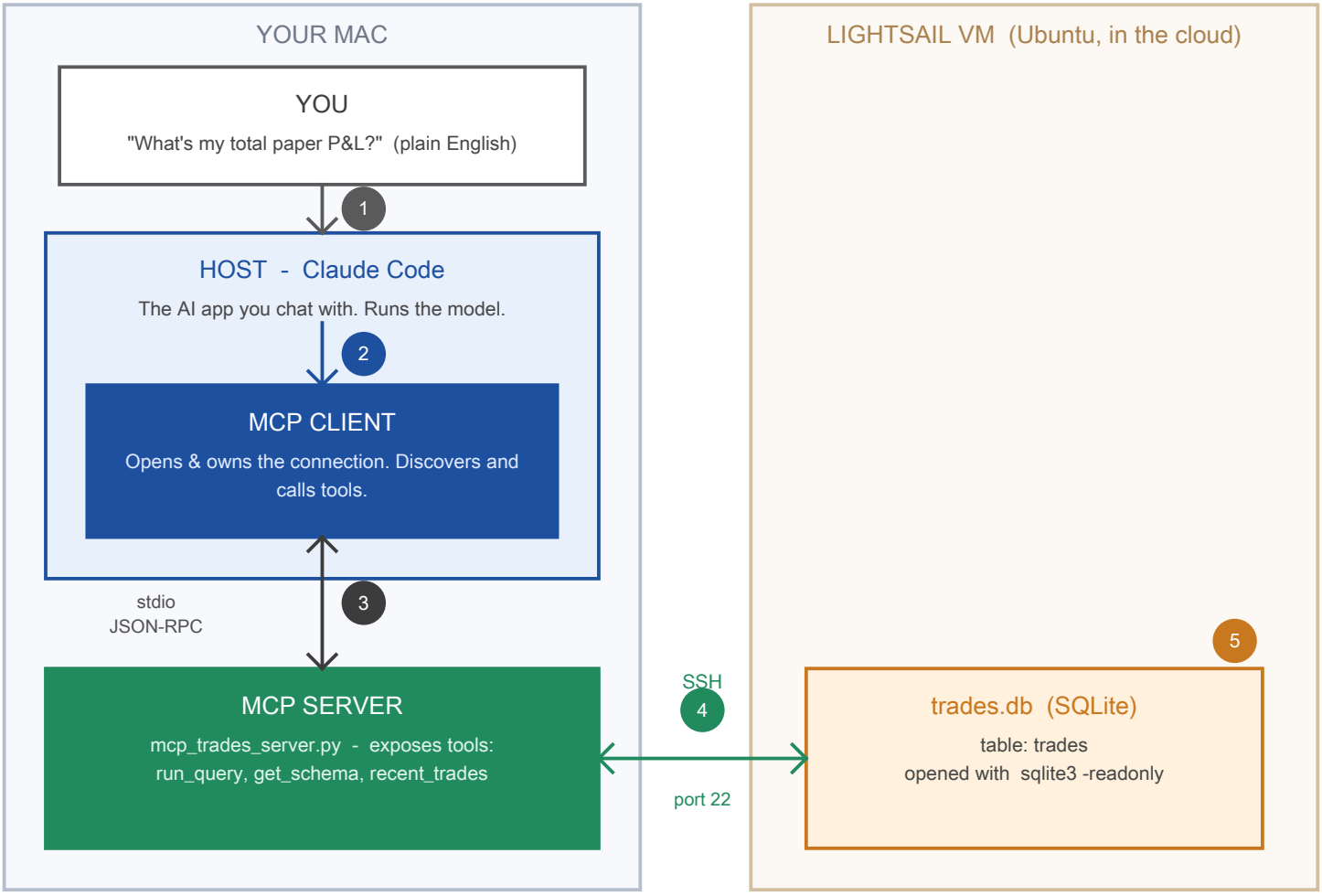
```
run_query(sql)              -> run a read-only SELECT, return rows
get_schema()               -> list the columns of the trades table
recent_trades(limit, mode) -> shortcut for the latest trades
```

How they connect: the transport

- stdio (local): client and server on the same machine talk over standard input/output. Simplest, nothing exposed to the network. This is what we use.
- HTTP / SSE (remote): the server runs as a web service the client reaches over the network. Needed when the server lives elsewhere.

3. How the data flows (our live example)

You ask a question in plain English; the answer comes from a SQLite database on a remote Ubuntu VM. Follow the numbered steps:



Steps 1-5 carry your request OUT to the database; the result retraces the same path BACK to you (step 6). The database is opened read-only, so the demo can never change real trade data.

You / Host MCP Client MCP Server Database

4. Walking the round trip

Each numbered step on the diagram, in order:

1. **You ask, in plain English**
   In Claude Code you type: "What's my total paper P&L?" No SQL, no commands.

2. **The model decides it needs data**
   The model (inside the Host) realises it should call a tool, and hands the request to the MCP Client.

3. **Client calls the server over stdio**
   The Client sends a structured 'tools/call' message (JSON-RPC) to the MCP Server over standard input/output. e.g. run_query('SELECT SUM(pnl) ...').

4. **Server reaches the VM over SSH**
   The Server first checks the SQL is read-only, then opens an SSH connection to the VM (orb-vm) and runs sqlite3 -readonly against trades.db.

5. **SQLite reads the database**
   On the VM, sqlite3 reads the 'trades' table read-only and returns the matching rows as JSON. The live data is never modified.

6. **The answer flows back**
   Rows travel back: VM -> Server -> (stdio) -> Client -> model. The model turns the rows into a plain-English answer: "Your total paper P&L is -Rs.10,699 across 7 trades."

Why route through SSH instead of opening the database to the internet?

The VM never exposes a database port. The server reaches it over the same secure SSH channel you already trust, and opens the file read-only - so a training demo can never corrupt real trade data.

5. Key terms for the session

Term	Plain-English meaning
MCP	Open standard for connecting AI assistants to tools & data.
Host	The AI app you use (Claude Code). Holds the model + clients.
Client	The connector inside the host that calls one server.
Server	A small program that exposes tools and does the work.
Tool	A callable function the model can invoke (e.g. run_query).
Resource	Read-only data a server offers for context.
Transport	How client & server talk: stdio (local) or HTTP (remote).
JSON-RPC	The structured message format they exchange.
stdio	Talking over standard input/output - same-machine, no network.

The one-line takeaway

An MCP CLIENT (inside your AI app) calls an MCP SERVER (a small program you control); the server does the real work - here, securely reading a database on a remote VM - and hands the result back to the model.